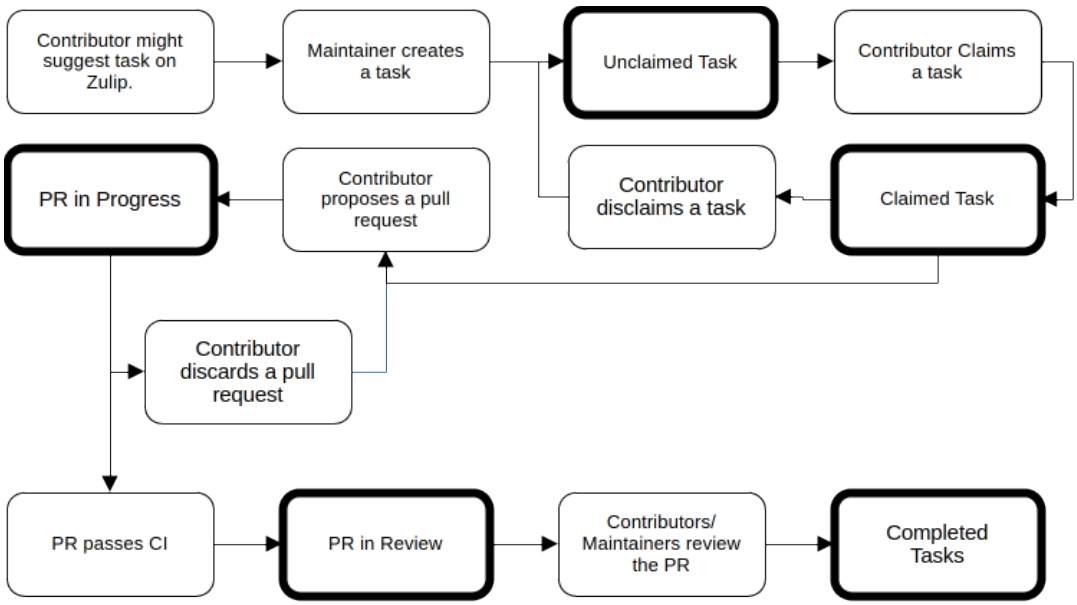

FIGURE 7. A partial flowchart of the automated task management process. Each task corresponds to an issue. A pull request is created to resolve tasks and once a pull request linked to a task is merged, the task is considered complete. The thick boxes represent states of the project dashboard represented by task columns. The movement of tasks between these states is automated by the CI which is triggered upon specific actions performed by contributors on the respective GitHub issues and pull requests. A more detailed description is found in the contributions file of the GitHub file, named `CONTRIBUTING.md` by convention.

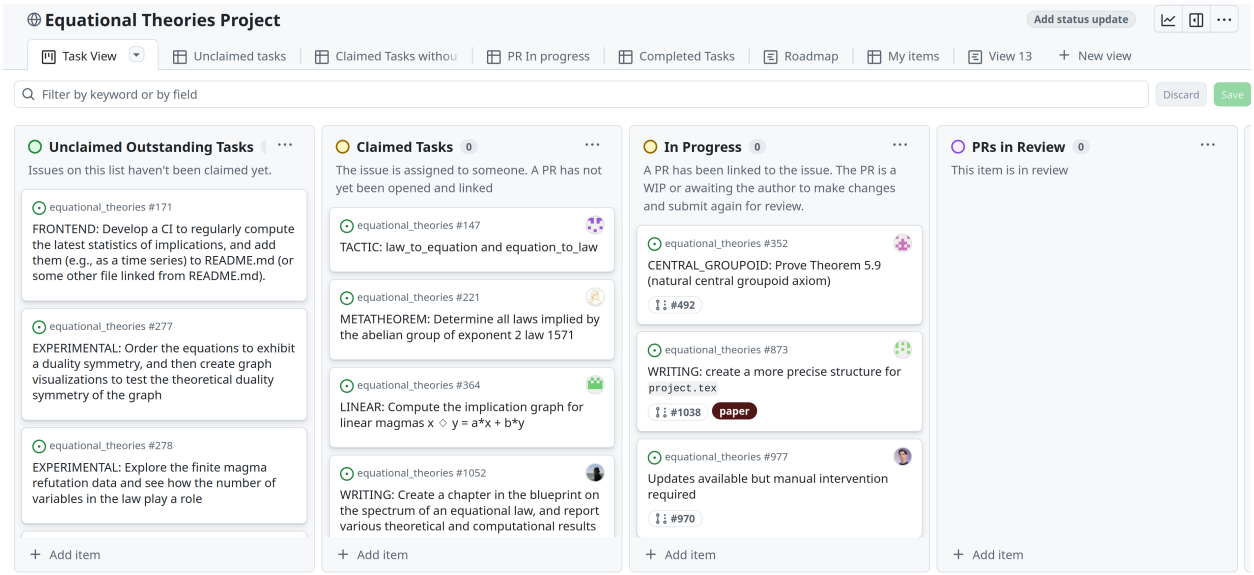

FIGURE 8. A snapshot of the project dashboard as of 30 July 2025

4.7. **Trusting ITPs to scale collaboration.** In our project we used the interactive theorem prover (hereon ITP) *Lean* 4 [53] precisely to address these issues of scaling. The contents of this section are common knowledge in the ITP and ITP-adjacent research communities. The exposition is intended to be useful to a user of ITPs.

At its core, an interactive theorem prover implements an expressive logic, encoded in a suitable choice of mathematical foundations. *Lean* 4 has the calculus of constructions extended by inductive types as its core logic. This logic is sufficiently powerful to express mathematical definitions and theorems for almost all areas of mathematical interest, while being relatively spartan and easy to write proof checkers for. Additionally, modern ITPs provide a convenient programming language which helps express mathematical ideas in a syntax closer to a mathematician's intuition than would be permitted by raw logical terms. A subset of ITPs like *Lean*, *Rocq* (formerly *Coq*), and *Isabelle* go one step further and provide the means to generate proofs through so-called *tactics*. There are usually numerous tactics, each specialised for specific proof generation methods. Among other things, they search mathematical libraries, simplify expressions, and identify lemmas and hypotheses to make progress in proofs. The proofs generated by this overlying programming machinery are terms in the core logical calculus which are checked mechanically by the proof checker. But in a large project, there is more to trust. It helps to understand the nature and limits of trust one can place on ITPs.

For our purposes, *Lean* consists of three pieces:

(1) A core proof checker called the *kernel*. This checker encodes a typed λ-calculus that is sufficiently expressive for most mathematical purposes. Without getting into details, theorems are encoded as a formal specification of the intended theorem statement. Proofs are encoded as deduction trees using known lemmata, acceptable axioms, and inference rules. The kernel checks whether this deduction tree constitutes a correct deduction in the context of existing theorems and lemmata, called the *environment*. Once a theorem's proof has been checked, it is added to the environment and can be used for constructing subsequent proofs. In reality, *Lean* allows users some flexibility in adding declarations into its environment without checking their types.
(2) A sophisticated programming language in which users express their definitions and theorems. Programs in this language are said to be *elaborated* to produce definitions and proofs in the core logic that the kernel can check.
(3) A compiler which compiles executable *Lean* code into reasonably efficient C code.

Of these three pieces, *the kernel is the smallest and most trusted piece*. Programs in the programming language are translated by an *elaborator* to the spartan language of the kernel. In reality the elaborator does much more, but the key takeaways are the following:

- The kernel only verifies programs of a very simple barebones language. A proof verified by *Lean* can be trusted *modulo* the correctness of the kernel implementation. This caveat can usually be dropped because the kernel is also one of the most battle-hardened pieces of an ITP.
- The kernel checks proofs against a given specification. This means that if the formal theorem statement itself is flawed or incorrectly uses other definitions, a correct and verified proof of this theorem would be mathematically meaningless. The formal

statement of a theorem gives expression to a mathematician's intuition and intention, and as such, the only check against mistakes in this area come from human review. *Lean* cannot offer any guarantees against false statements written by its users or artificial intelligence tools.

- The higher level programming language can in-principle, generate arbitrary deductions. Their correctness is ultimately validated by the kernel. This gives the higher level programming language more leeway in generating possibly incorrect deductions.
- Ideally the environment only consists of definitions and lemmata already checked by the kernel. Thus their correct usage in the proof of subsequent theorems is valid. However this assumption is often not entirely true in practical implementations of theorem provers for efficiency reasons. In this case, trust in the kernel is restored by replaying each assumption from scratch in the kernel. This can be accomplished by external checkers. Such checkers, which are ideally independent implementations of the *kernel*, can also guard against implementation mistakes in the kernel. In this project we used the environment replay tool *lean4checker* and the *lean4lean* [16] tool. To our knowledge this was the first such use of the *lean4lean* tool.

Remark 4.1. A bug in the kernel that allows false statements to be proved is usually called a *soundness* bug. Concretely, a soundness bug can be exploited to produce a kernel-certified proof of the proposition `False`. Such bugs are rare but not entirely non-existent. This is distinct from being able to prove `False` by simply assuming contradictory or false statements. A proof of the proposition `False` implies a proof of any statement by *ex falso quod libet*.

Users of *Lean* only interact directly with the higher level programming language and usually get confirmation of the correctness of their proofs through the editor interface. Further, collaborators on a project such as ours are likely to deploy automated tools such as SAT solvers, SMT solvers, first-order theorem provers, and perhaps even modern AI tools. These tools usually produce proof certificates which are imported or inserted into a *Lean* source file. Some modern AI tools are integrated into code editors which might automatically produce or edit even the statements of theorems and the definitions they deal with. Given the limits to trust mentioned above, a productive and useful collaboration using *Lean* also requires a collaboration and verification infrastructure combining human effort and automated tools. It is in this context that we discuss the project infrastructure. It is a concrete answer to the questions posed by one of the authors at the beginning of this project [65] that combines tools from the ITP community and the software engineering community. All these tools already exist. The goal of this exposition is to explain how they address the concerns described above.

The non-Lean pieces: While *Lean* can check proofs of theorems up to the limitations described above, a project of this scale involves the use of several non-*Lean* tools. For example there are tools which extract the proven implications and anti-implications. There are tools which construct various visualisations. There are also metaprograms which call external automated theorem provers, and extract proof certificates from them to construct a *Lean* proof out of them. In keeping with the garbage-in garbage-out principle, if these tools get spurious inputs and throw spurious outputs, *Lean* can only tell us that the proofs are incorrect after the formal proofs have been translated to *Lean*. It cannot, for instance, stop us from generating a large number of spurious conjectures that misguide contributors